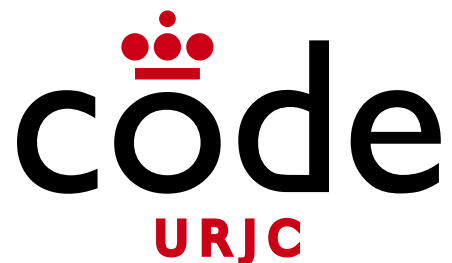


Desarrollo Web

Tema 4.10:

Seguridad en Spring Web + REST



©2026

Micael Gallego, Francisco Gortázar, Michel Maes, Óscar Soto, Iván Chicano

Algunos derechos reservados

Este documento se distribuye bajo la licencia
“Atribución-CompartirIgual 4.0 Internacional”
de Creative Commons Disponible en
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Aplicaciones web y REST

rest_ejem17_security_with_web

- Es posible que una misma aplicación ofrezca un **interfaz web y una API REST**
- Para ello, se usan las dos configuraciones de `SecurityFilterChain`
- Hay que hacer ciertos ajustes para que puedan coordinarse
 - Tendremos que cambiar el de REST
 - El de web se mantendrá igual que en otros ejemplos

Aplicaciones web y REST

rest_ejem17_security_with_web

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    ...

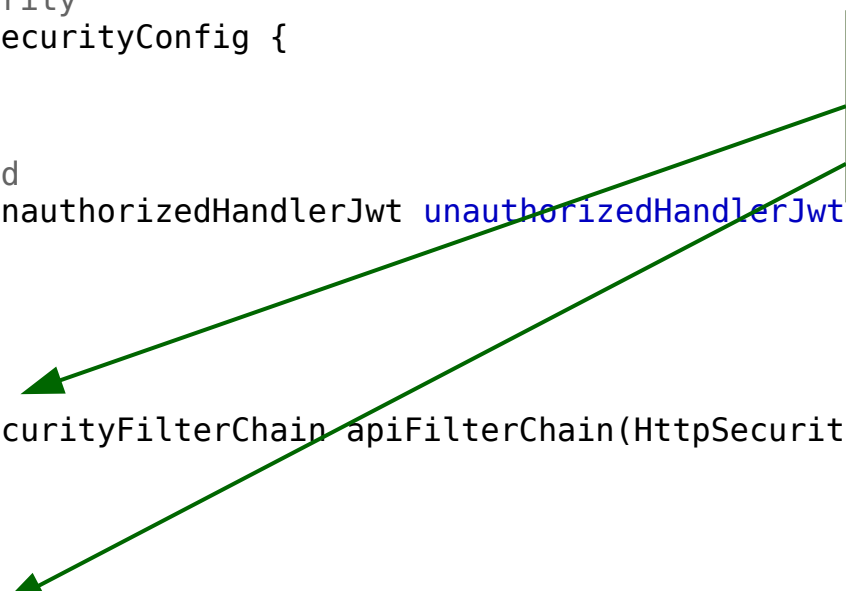
    @Autowired
    private UnauthorizedHandlerJwt unauthorizedHandlerJwt;

    ...

    @Bean
    @Order(1)
    public SecurityFilterChain apiFilterChain(HttpSecurity http) throws Exception {
        ...
    }

    @Bean
    @Order(2)
    public SecurityFilterChain webFilterChain(HttpSecurity http) throws Exception {
        ...
    }

}
```



La configuración para REST se carga la primera

Aplicaciones web y REST

rest_ejem17_security_with_web

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    ...

    @Autowired
    private UnauthorizedHandlerJwt unauthorizedHandlerJwt;

    ...

    @Bean
    @Order(1)
    public SecurityFilterChain apiFilterChain(HttpSecurity http) throws Exception {
        ...
    }

    @Bean
    @Order(2)
    public SecurityFilterChain webFilterChain(HttpSecurity http) throws Exception {
        ...
    }
}
```

Necesitaremos este handler para que los errores en REST no se redirijan a WEB

Aplicaciones web y REST

rest Ejem17_security_with_web

```
@Component
public class UnauthorizedHandlerJwt implements AuthenticationEntryPoint {

    private static final Logger logger = LoggerFactory.getLogger(UnauthorizedHandlerJwt.class);

    @Override
    public void commence(HttpServletRequest request, HttpServletResponse response,
        AuthenticationException authException)
        throws IOException, ServletException {
        logger.error("Unauthorized error: {}", authException.getMessage());

        response.setContentType(MediaType.APPLICATION_JSON_VALUE);
        response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);

        final Map<String, Object> body = new HashMap<>();
        body.put("status", HttpServletResponse.SC_UNAUTHORIZED);
        body.put("error", "Unauthorized");
        body.put("message", authException.getMessage());
        body.put("path", request.getServletPath());

        final ObjectMapper mapper = new ObjectMapper();
        mapper.writeValue(response.getOutputStream(), body);
    }
}
```

Crear  una respuesta custom (JSON) ante cualquier excepci n de autenticaci n/autorizaci n en REST

Aplicaciones web y REST

```
@Bean
@Order(1)
public SecurityFilterChain apiFilterChain(HttpSecurity http) throws Exception {

    http.authenticationProvider(authenticationProvider());

    http
        .securityMatcher("/api/**")
        .exceptionHandling(handling -> handling.authenticationEntryPoint(unauthorizedHandlerJwt));

    http
        .authorizeHttpRequests(authorize -> authorize
            // PRIVATE ENDPOINTS
            .requestMatchers(HttpMethod.POST, "/api/books/").h
            .requestMatchers(HttpMethod.PUT, "/api/books/**").
            .requestMatchers(HttpMethod.DELETE, "/api/books/**
            // PUBLIC ENDPOINTS
            .anyRequest().permitAll()
        );

    // Disable Form login Authentication
    http.formLogin(formLogin -> formLogin.disable());

    // Disable CSRF protection (it is difficult to implement in REST APIs)
    http.csrf(csrf -> csrf.disable());

    // Disable Basic Authentication
    http.httpBasic(httpBasic -> httpBasic.disable());

    // Stateless session
    http.sessionManagement(management -> management.sessionCreationPolicy(SessionCreationPolicy.STATELESS));

    // Add JWT Token filter
    http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);

    return http.build();
}
```

rest Ejem17_security_with_web

Para evitar colisiones, la configuración de método sólo se aplica a las URLs que empiezan por /api

Aplicaciones web y REST

```
@Bean
@Order(1)
public SecurityFilterChain apiFilterChain(HttpSecurity http) throws Exception {

    http.authenticationProvider(authenticationProvider());

    http
        .securityMatcher("/api/**")
        .exceptionHandling(handling -> handling.authenticationEntryPoint(unauthorizedHandlerJwt));

    http
        .authorizeHttpRequests(authorize -> authorize
            // PRIVATE ENDPOINTS
            .requestMatchers(HttpMethod.POST, "/api/books/").has
            .requestMatchers(HttpMethod.PUT, "/api/books/**").ha
            .requestMatchers(HttpMethod.DELETE, "/api/books/**")
            // PUBLIC ENDPOINTS
            .anyRequest().permitAll()
        );

    // Disable Form login Authentication
    http.formLogin(formLogin -> formLogin.disable());

    // Disable CSRF protection (it is difficult to implement in REST APIs)
    http.csrf(csrf -> csrf.disable());

    // Disable Basic Authentication
    http.httpBasic(httpBasic -> httpBasic.disable());

    // Stateless session
    http.sessionManagement(management -> management.sessionCreationPolicy(SessionCreationPolicy.STATELESS));

    // Add JWT Token filter
    http.addFilterBefore(jwtRequestFilter, UsernamePasswordAuthenticationFilter.class);

    return http.build();
}
```

rest Ejem17_security_with_web

Damos de alta el handler creado para que se ocupe de las excepciones REST (y no las redirija a WEB)